

FT8 Local DX Spotter — Desktop Application

Project Brief

Project Goal

Build a desktop application that watches current FT8 reception reports and alerts me when amateur-radio stations near my location are hearing long-distance DX.

This is not intended to be a general worldwide DX cluster. The important question is:

“Are stations reasonably close to me currently hearing DX that I may also be able to hear and contact?”

For example, if several stations within 100 miles of my location are hearing FT8 stations in Europe on 20 meters, that is useful evidence that I should check 20 meters.

My amateur-radio callsign is **KE0CGB**, and I am located in the **Jerico Springs/Cedar County area of southwest Missouri**. The app must also allow location and callsign settings to be changed.

Platform and Interface

Build this as a normal desktop application.

- Target Windows 11 first.
- Keep the underlying code reasonably cross-platform for possible Linux and macOS support.
- The app must be able to minimize to the system tray and continue monitoring in the background.
- Use a normal desktop layout and normal-sized fonts.
- Do not use giant, bold kiosk-style text.
- Text scaling or font size may be offered as an optional setting, but the default should suit a normal 1440p desktop monitor.
- Use a clean dark theme if practical.
- The app should not require a web server or browser to operate.

A good technical choice would be Python with PySide6, unless research shows another framework is materially better. Keep data collection, DX analysis, alerts, settings, and the graphical interface separated into modules.

Core Operating Principle

The app should:

1. Know the user’s location from a Maidenhead grid square, latitude/longitude, or callsign lookup.
2. Retrieve recent FT8 reception reports from PSK Reporter or another legitimate source.

3. Identify reporting/receiving stations located within a user-selected radius of the user.
4. Examine the distant stations those nearby receivers are hearing.
5. Determine whether the reports indicate a useful DX opening that the user may also be able to hear.
6. Display the results and optionally issue a spoken Windows notification.

The app must distinguish between:

- The station transmitting the FT8 signal.
- The receiving station reporting that signal.
- The distance from the user to the receiving station.
- The distance from the user or receiver to the transmitting DX station.

The fact that a distant station appears somewhere on PSK Reporter is not enough. At least one receiver near the user must have reported hearing it.

User-Configurable Settings

Include settings for:

- User callsign.
- Maidenhead grid square or latitude/longitude.
- Nearby receiver radius in miles.
- Minimum DX distance.
- Report age window.
- Bands to monitor.
- Continents or regions of interest.
- Minimum number of nearby receivers hearing similar DX.
- Minimum acceptable signal-to-noise ratio, if desired.
- Voice alerts on or off.
- Desktop notifications on or off.
- Alert cooldown.
- Launch with Windows.
- Start minimized.
- Distance units: miles or kilometers.

Suggested defaults:

- Nearby receiver radius: 100 miles.
- Selectable range: approximately 25–500 miles.
- Minimum DX distance: 1,500 miles.
- Recent report window: 10 minutes.
- Alert cooldown: 10 minutes.
- Monitor the common FT8 HF bands by default.

These values must remain editable.

Main Window

The main window should provide an immediate picture of useful DX activity.

Suggested sections:

Current Band Conditions

Show each selected band with a simple status such as:

- Quiet
- Some DX
- Good opening
- Strong opening

The rating must be based on actual nearby-receiver reports, not a random propagation estimate.

Current DX Results

Use a sortable table with columns such as:

- Time heard
- FT8 band
- DX callsign
- DX grid
- Country or entity
- Continent
- Distance
- Receiving station
- Receiver distance from me
- SNR
- Number of nearby receivers hearing it
- Reason the report was considered notable

Normal table fonts should be used. Avoid excessively large cards that waste screen space.

Summary Area

Examples:

- "20 meters: Good opening to Europe"
- "Three nearby receivers have heard Germany during the last 8 minutes"
- "40 meters: South America activity increasing"
- "17 meters: One nearby receiver heard Japan at -15 dB"

Status Area

Show:

- Last successful update.
- Next scheduled update.
- Number of reports retrieved.
- Number of nearby receiving stations found.
- Current API status.
- Any rate-limit or connection error.

DX Qualification and Scoring

Create a transparent scoring system. Do not hide everything behind an unexplained AI judgment.

Possible scoring factors:

- Number of nearby receivers hearing the same station, country, or region.
- How close each receiving station is to the user.
- Age of the reports.
- Signal strength/SNR.
- DX distance.
- Whether several different DX stations from the same region are being heard.
- Whether activity is increasing.
- Whether reports appear on more than one nearby receiver.
- Whether the band and direction make geographical sense.

Reports from several independent nearby stations should carry more weight than many duplicate reports from one station.

An opening should also be recognized when different nearby receivers are hearing different stations from the same country, continent, or general region.

Display the reason for the score. Examples:

- "Four receivers within 72 miles are hearing Europe."
- "Two nearby stations heard the same Japanese station."
- "European reports on 20 meters increased sharply in five minutes."

Voice and Desktop Alerts

The application should be useful while minimized.

Example voice alerts:

- "DX alert. Twenty meters is opening to Europe."
- "Nearby stations are hearing Japan on seventeen meters."

- “Strong South America activity reported on fifteen meters.”
- “Three stations near you are hearing Germany and France.”

Do not read every callsign individually unless the user enables that option.

Prevent repetitive announcements:

- Group related reports.
- Apply a cooldown by band and region.
- Do not repeat an alert unless the opening becomes stronger, a new important region appears, or the cooldown expires.
- Allow quiet hours.
- Include a Test Voice button.
- Let the user choose from available system voices if practical.

PSK Reporter/API Research Requirements

Before building the full interface, research the current official and permitted PSK Reporter data-access methods.

Do not assume an endpoint, query parameter, response field, or polling rate without verifying it.

The first development milestone must be a small standalone proof-of-concept that:

1. Sends a valid request.
2. Receives current FT8 data.
3. Saves or displays a sample response.
4. Correctly identifies the transmitting and receiving callsigns.
5. Extracts timestamps, frequencies, grids, modes, and SNR values where available.
6. Converts frequency to an amateur band.
7. Demonstrates that the request can be repeated without violating rate limits.

Only after this works should it be connected to the graphical interface.

Follow these rules:

- Prefer documented APIs or feeds over scraping web pages.
- Respect PSK Reporter’s acceptable-use requirements and request limits.
- Use a descriptive User-Agent if requested by the service.
- Cache results locally.
- Avoid requesting the same data unnecessarily.
- Use exponential backoff after errors.
- Use UTC internally for all timestamps.
- Log requests, response status, number of records, and parsing errors.
- Do not hammer the service while debugging the UI.
- Provide an offline test mode using saved response fixtures.

- If the main data source is unavailable, show that clearly rather than displaying stale information as current.
- Do not silently substitute fabricated or demonstration data.

If PSK Reporter cannot efficiently answer the needed query directly, research a compliant method of collecting a recent report window and filtering it locally.

Document exactly what endpoint or feed was selected, why it was selected, expected request frequency, and known limitations.

Location and Callsign Handling

Maidenhead grid coordinates should be converted to approximate latitude and longitude locally.

Use a standard great-circle or Haversine calculation for distance.

Callsign-to-country/entity information may come from a maintained amateur-radio prefix/entity dataset. Avoid making a paid QRZ subscription mandatory.

The program should still work if only callsign and grid information are available.

For portable or compound callsigns, make a reasonable attempt to identify the actual entity, but retain the original reported callsign in the display.

Data Processing Rules

- Normalize callsigns to uppercase.
- Ignore malformed records without crashing.
- Deduplicate repeated spots.
- Retain the strongest or newest useful observation where appropriate.
- Keep observations from separate receiving stations distinct.
- Convert frequency to band using a clearly defined table.
- Expire old records when they leave the selected time window.
- Keep a short in-memory history to detect whether activity is rising or falling.
- Store user settings locally.
- Do not permanently accumulate millions of reception reports.
- Make retention configurable if a history feature is added.

Maps and Extra Features

A map is useful but not required for the first working version.

A later map could show:

- My station.
- Nearby reporting receivers.

- DX transmitters.
- Lines from the receivers to the DX stations.
- Filters by band, continent, age, and signal strength.

Do not allow a complicated map to delay the core monitoring and alert system.

Other possible later features:

- Activity history by band and region.
- Opening trend charts.
- “Who near me is hearing this DX?” view.
- Worked-before filtering using an ADIF log.
- Wanted-country or wanted-grid alerts.
- WSJT-X integration.
- Button to switch WSJT-X to the recommended band.
- User-defined watch lists.
- POTA/SOTA identification.
- Solar and propagation information as secondary context.

These are later enhancements, not first-version requirements.

Privacy and Safety

- No transmitter control is needed in the first version.
- The application must never automatically transmit.
- Do not upload the user’s log or precise location without clear permission.
- Store settings locally.
- If third-party lookup services are optional, explain what information is sent to them.
- Do not require API keys unless the selected service genuinely requires one.

Development Order

Phase 1: Data Proof-of-Concept

- Verify the PSK Reporter data source.
- Retrieve current FT8 reports.
- Parse and normalize them.
- Identify nearby receivers.
- Calculate distances.
- Print a list of DX being heard by those receivers.
- Save sample responses for offline testing.

Phase 2: Analysis Engine

- Add band detection.
- Add filters.
- Add deduplication.
- Add region/entity identification.

- Add DX confidence scoring.
- Add alert grouping and cooldown logic.
- Write unit tests using saved reports.

Phase 3: Desktop Interface

- Main results table.
- Band-condition summary.
- Settings window.
- Status and error reporting.
- Refresh controls.
- Normal desktop-sized fonts and spacing.

Phase 4: Background Operation

- System tray.
- Voice announcements.
- Windows notifications.
- Start minimized.
- Optional launch at login.
- Graceful network-error recovery.

Phase 5: Packaging

- Package as an easy Windows installer or standalone application.
- Do not require the user to manually install Python.
- Include application version information.
- Include a readable log file and a way to open it from the Help menu.
- Include a short setup guide.

Development Rules for Antigravity

1. Do not attempt to build the entire application before proving that live data retrieval works.
2. Do not repeatedly redesign the interface while the data source is still unverified.
3. Do not invent API behavior or response fields.
4. Show the actual tested request and a redacted sample response.
5. Keep a saved fixture so development can continue without hammering PSK Reporter.
6. Separate the data provider from the rest of the program so it can be replaced later.
7. Keep scoring rules understandable and configurable.
8. Handle empty results as a normal condition.
9. Make errors visible and useful.
10. Maintain a progress document showing completed, current, and blocked tasks.
11. Maintain a change log.
12. Run tests after each meaningful change.
13. Do not remove working functionality while adding a new feature.
14. If blocked, state the exact technical reason instead of repeatedly trying unrelated changes.
15. Deliver a small working version before adding maps, propagation predictions, CAT control, or other advanced features.

Minimum Viable Product Acceptance Test

The first usable version is complete when it can:

- Accept my location and nearby radius.
- Retrieve real recent FT8 reception reports.
- Identify receiving stations near me.
- Show distant stations those receivers are hearing.
- Filter reports by band, distance, age, and region.
- Explain why an opening is considered noteworthy.
- Run in the Windows system tray.
- Speak a useful alert without excessive repetition.
- Recover cleanly from loss of internet or API errors.
- Save settings between runs.
- Provide logs that make troubleshooting possible.
- Install and run on Windows 11 without requiring a development environment.

The primary measure of success is not how many spots the app displays. It is whether the app gives me a useful and timely indication that I should turn on the radio and check a particular FT8 band because stations near me are currently hearing workable DX.